Instituto Tecnológico de Mazatlán

Driver de software para un relevador: modelado de relevadores y puentes H con Arduino UNO R4 WiFi

Carrera:
Ingeniería en Sistemas Computacionales

Materia:
Sistemas Programables

Profesor:
Miguel Barrón Hernández

Integrantes:
Ximena Armenta Dávila
Carlos Tadeo Ibarra Castañeda
Dulce Princesa Ulivarría González

1. Introducción

Un pin del Arduino UNO R4 WiFi entrega muy poca corriente (la guía de la práctica menciona apenas unos 8 mA), mientras que un motor pequeño puede pedir cientos de miliamperes al arrancar. Por eso el Arduino solo da la orden: la energía sale de otra fuente y un dispositivo intermedio, como un relevador o un puente H, es el que conmuta la carga.

Un relevador real tiene partes mecánicas que se desgastan y tardan en moverse, así que no debe conmutarse demasiado rápido. Además, hay dispositivos que se activan con nivel alto y otros con nivel bajo, como el optoacoplador de un módulo de relevador. En la industria no se maneja a este intermediario con pines sueltos repartidos por el programa: se construye un modelo de software del dispositivo y el resto del programa le habla con órdenes que tienen significado, como encender y apagar.

En esta práctica se programó ese modelo como una clase en C++ llamada `Relevador`, que recuerda el estado del actuador, traduce la orden al nivel de pin correcto, hace cumplir un tiempo mínimo de 1000 ms entre cambios y deja todo apagado al iniciar. Como no se cuenta con un módulo de relevadores, se usaron dos actuadores que lo representan: un canal del puente H L298N con un motorreductor, que se activa con nivel alto, y un LED conectado “al revés”, que se activa con nivel bajo. Una sola clase controla ambos desde el Monitor Serie, con `millis()` y sin usar `delay()`.

2. Capturas del circuito funcionando

En la Figura 1 se observa el armado: la placa está conectada a la computadora por USB, el LED y su resistencia están en la protoboard y los cables del motorreductor llegan al puente H L298N. En la Figura 2 se muestra el Monitor Serie durante las pruebas, con las respuestas a los comandos M1, M0, L1, L0 y E.

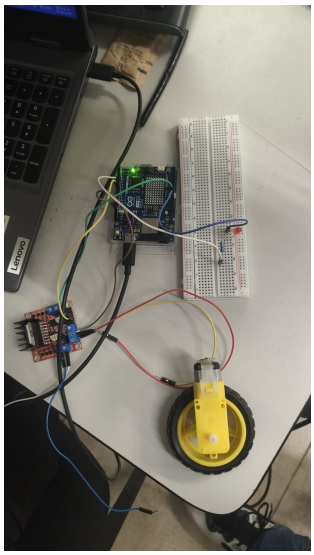


Figura 1. Armado físico con el Arduino UNO R4 WiFi, la protoboard con el LED y la resistencia, el puente H L298N y el motorreductor con rueda.

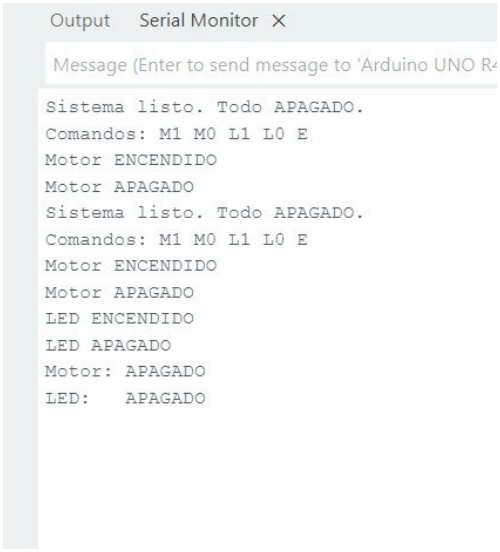


Figura 2. Monitor Serie con las órdenes del motor y del LED y la consulta de estado.

3. Metodología

Primero se armó el circuito con el USB desconectado y se pidió al docente que lo revisara antes de energizar. La parte de potencia se separó de la de señal: el positivo del portapilas va a la terminal +12V del L298N, el negativo a su terminal GND y el motorreductor a las salidas OUT1 y OUT2. El GND del L298N se unió al GND del Arduino para tener una tierra común, que es obligatoria, y se dejaron puestos los jumpers de ENA y 5V-EN. Del lado de la señal, D7 se conectó a IN1 del L298N e IN2 se llevó a GND. El LED se conectó al revés: el ánodo (pata larga) va a 5V y el cátodo pasa por la resistencia de 220 Ω hasta D6.

Se tuvo cuidado de no alimentar el motor desde el pin 5V del Arduino, de no conectar la terminal +5V del L298N al Arduino y de revisar la polaridad del portapilas antes de conectar.

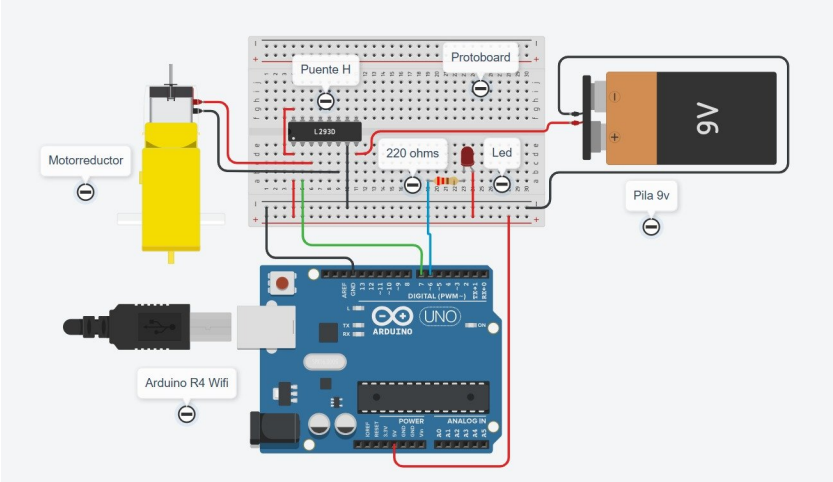


Figura 3. Circuito simulado en Tinkercad (en la simulación el puente H es un L293D y la fuente del motor una pila de 9 V).

Tabla I. Conexiones del circuito con el Arduino UNO R4 WiFi.

Elemento / pin	Conexión	Función
Portapilas (+)	+12V del L298N	Alimentación de potencia del motor.
Portapilas (−)	GND del L298N	Tierra de la parte de potencia.
GND del L298N	GND del Arduino	Tierra común, obligatoria.
Motorreductor	OUT1 y OUT2 del L298N	Salida de potencia al motor.
IN1 del L298N	D7	Orden del motor (activo en alto).
IN2 del L298N	GND	Fija el sentido de giro.
Ánodo del LED	5V	Alimentación del LED.
Cátodo del LED	220 Ω → D6	Orden del LED (activo en bajo).

Antes de programar se dibujó el diagrama de estados del relevador (Figura 4). El estado inicial es APAGADO y una orden solo cambia el estado si ya pasó el tiempo mínimo desde el último cambio; si

llega antes, se rechaza y el pin no cambia.

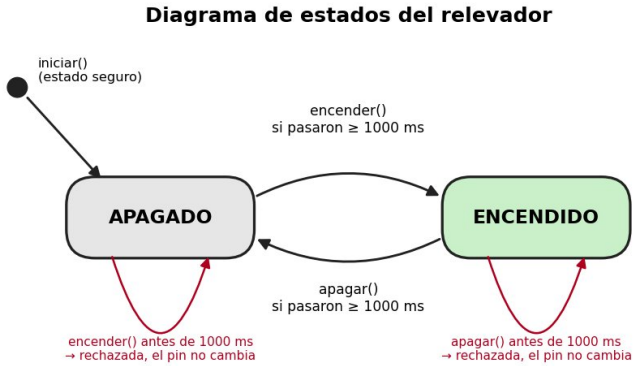


Figura 4. Diagrama de estados del relevador con el estado inicial y la regla del tiempo mínimo de 1000 ms.

El programa (`Practica_4_1_1_A.ino`) se organiza de la siguiente manera:

- **Datos privados de la clase:** el pin que controla, si es activo en bajo o en alto, el estado actual, el momento del último cambio medido con `millis()` y el tiempo mínimo entre cambios (1000 ms).
- **Traducción de nivel:** el método privado `_nivelPara()` es el único lugar donde se decide si se escribe HIGH o LOW. Un actuador activo en alto se enciende con HIGH y uno activo en bajo con LOW, así que el resto del programa solo habla de “encender” y “apagar”.
- **Cambio de estado:** `_cambiarA()` regresa `true` sin conmutar si el actuador ya está en el estado pedido, regresa `false` sin tocar el pin si todavía no pasa el tiempo mínimo, y en caso contrario escribe el nivel, guarda el nuevo estado y reinicia el momento del último cambio. `encender()` y `apagar()` solo llaman a este método.
- **Estado seguro:** `iniciar()` escribe el nivel de apagado antes de configurar el pin como salida y otra vez después. Así un actuador activo en bajo, como el LED, no recibe un pulso de encendido durante el arranque.
- **Dos objetos de la misma clase:** uno para el motor (D7, activo en alto) y otro para el LED (D6, activo en bajo).
- **Comandos por el Monitor Serie:** `leerSerie()` arma cada línea hasta recibir el salto de línea y la pasa a mayúsculas; `procesar()` la compara con M1, M0, L1, L0 y E, e `intentar()` responde con “Motor ENCENDIDO”, “LED APAGADO” u “Orden rechazada: espere X ms entre cambios”, donde X lo calcula `faltaMs()`.

```
bool _cambiarA(bool destino) {
  if (_encendido == destino) return true;   // ya esta ahi, no conmuta
  if (millis() - _ultimoCambio < _tMinimo) return false;  // rechazada
  digitalWrite(_pin, _nivelPara(destino));
  _encendido    = destino;
  _ultimoCambio = millis();
  return true;
}

Relevador motor(7, false);   // L298N IN1 - activo en ALTO
Relevador led  (6, true);    // LED al reves - activo en BAJO
```

El programa principal nunca llama a `digitalWrite()` y no usa `delay()`: el `loop()` solo ejecuta `leerSerie()`, por lo que nunca se bloquea. Al iniciar, el programa abre el puerto serie a 9600 baudios y espera hasta 3 s a que se abra el Monitor Serie para no perder los primeros mensajes.

Tabla II. Comandos del Monitor Serie.

Comando	Acción
M1 / M0	Encender o apagar el motor.
L1 / L0	Encender o apagar el LED.
E	Mostrar el estado de ambos.

4. Análisis de resultados

Respuesta del sistema a las órdenes

La Tabla III resume lo que se observó en el Monitor Serie. Aunque el motor se enciende con nivel alto y el LED con nivel bajo, desde el programa principal se mandan las mismas órdenes a los dos objetos, porque la traducción de nivel la hace la clase.

Tabla III. Órdenes probadas y respuestas observadas en el Monitor Serie.

Orden	Respuesta observada	Interpretación
Arranque o RESET	“Sistema listo. Todo APAGADO.” y la lista de comandos	Estado seguro: el motor y el LED quedan apagados.
M1	Motor ENCENDIDO	La clase escribe HIGH en D7 (activo en alto).
M0	Motor APAGADO	La clase escribe LOW en D7.
L1	LED ENCENDIDO	La clase escribe LOW en D6 (activo en bajo).
L0	LED APAGADO	La clase escribe HIGH en D6.
E	Motor: APAGADO / LED: APAGADO	Se muestra el estado de ambos objetos.

El mensaje de arranque aparece dos veces en la captura porque la placa se reinició, y en cada arranque todo quedó apagado. Los comandos se pasan a mayúsculas antes de compararse, así que “m1” funciona igual que “M1”.

Pruebas de seguridad y de inversión

- **Prueba de seguridad 1 (M1 y M0 muy seguidos):** la segunda orden llega antes de 1000 ms, `_cambiarA()` regresa `false` y el sistema responde “Orden rechazada: espere X ms entre cambios”. El pin no cambia, por lo que el motor sigue en su estado. Como `iniciar()` también guarda el momento del arranque, una orden enviada en el primer segundo después de encender o reiniciar igualmente se rechaza.

- **Prueba de seguridad 2 (RESET con el motor encendido):** al reiniciar, el Arduino vuelve a ejecutar `setup()` e `iniciar()`, y el motor y el LED quedan apagados sin importar cómo estuvieran antes.
- **Prueba de inversión:** si el LED se crea por error como "activo en alto", `encender()` escribiría HIGH y el LED se apagaría, y `apagar()` escribiría LOW y el LED se encendería. Desde el arranque quedaría prendido, aunque el Monitor Serie dijera "APAGADO", porque el Monitor muestra el estado que la clase cree tener, no el nivel eléctrico del pin.

Observaciones sobre el comportamiento del sistema:

- Una orden hacia el estado en el que ya está el actuador no cuenta como cambio: la clase responde que se realizó, pero no conmuta ni reinicia el tiempo mínimo.
- Toda la regla del tiempo mínimo está en la clase, por lo que el motor y el LED quedan protegidos igual sin repetir código en el `loop()`.
- Para que el motor responda a la señal de D7, el GND del L298N debe estar unido al GND del Arduino; sin esa referencia común el L298N no interpreta correctamente el nivel de IN1.
- Con `millis()` y sin `delay()`, el programa atiende los comandos apenas llegan, incluso mientras el tiempo mínimo de un actuador todavía no se cumple.

5. Preguntas de reflexión

1. ¿Por qué el LED se enciende con LOW y no con HIGH? Explica el recorrido de la corriente.

El ánodo del LED está conectado a 5V y el cátodo va a D6 a través de la resistencia de 220 Ω. Cuando D6 está en LOW (0 V) hay diferencia de voltaje, y la corriente sale de 5V, pasa por el ánodo, el LED, el cátodo y la resistencia, y entra al pin D6, que la manda a tierra, por eso el LED se enciende. Cuando D6 está en HIGH (5 V), los dos lados tienen casi el mismo voltaje, no hay diferencia y no circula corriente, así que el LED se apaga.

2. ¿Qué ventaja tiene que la regla del tiempo mínimo viva dentro de la clase y no en el loop()?

Que la regla se cumple siempre, porque ninguna orden puede saltársela: todo pasa por encender() y apagar(). Además se escribe una sola vez y sirve para todos los objetos (motor y LED), y si se quiere cambiar el tiempo solo se modifica en la clase. El loop() queda sencillo y sin lógica de seguridad repetida.

3. ¿Por qué un relevador real no sirve para controlar velocidad con PWM?

Porque el PWM conmuta miles de veces por segundo y un relevador tiene contactos mecánicos que tardan en moverse y se desgastan con cada cambio. No alcanzaría a seguir la señal y se dañaría muy rápido. El relevador solo sirve para encender y apagar; para controlar velocidad se usa un transistor o un puente H con PWM.

4. ¿Por qué el estado seguro es “apagado”? Da un ejemplo industrial donde convenga lo contrario (contacto NC).

Porque si se va la luz o se reinicia el sistema, la carga no debe arrancar sola: un motor que se enciende de repente puede lastimar a alguien o dañar la máquina. Un ejemplo contrario es una bomba de enfriamiento o un freno de seguridad: con un contacto normalmente cerrado (NC), si falla la energía o el control, el contacto sigue cerrado y la bomba sigue funcionando para evitar un sobrecalentamiento.

5. ¿Por qué es obligatoria la tierra común entre el portapilas y el Arduino?

Porque el voltaje siempre se mide respecto a una referencia. La señal de D7 está referida al GND del Arduino y el L298N interpreta IN1 respecto a su propio GND. Si no están unidos, el L298N no tiene la misma referencia, no sabe si recibe HIGH o LOW y el motor funcionaría mal o no funcionaría.

6. Si mañana cambiaras el L298N por un módulo de relevador real, ¿qué partes del programa tendrías que modificar?

Casi nada. Solo la línea donde se crea el objeto (el pin y si es activo en alto o en bajo, porque muchos módulos de relevador son activo en bajo) y, si hace falta, el tiempo mínimo. La clase y el resto del programa siguen igual, porque la traducción HIGH/LOW está encapsulada dentro de la clase.

6. Conclusiones individuales

Ximena

En esta práctica entendí que un relevador se puede ver como un objeto con estado y reglas, y no solo como un pin que se prende y se apaga. Poner el tiempo mínimo dentro de la clase hace que el actuador quede protegido aunque el programa principal mande órdenes muy seguidas, y sin usar delay() el sistema nunca se bloquea.

Carlos Tadeo

Lo más importante que vi fue que la misma clase sirve para el motor y para el LED aunque uno sea activo en alto y el otro en bajo, porque esa diferencia se resuelve en un solo método. La prueba de inversión me ayudó a entender qué pasaría si se marca mal un actuador: el Monitor Serie diría una cosa y el LED haría la contraria.

Dulce Princesa

Con esta práctica vi cómo encapsular el hardware en una clase hace el programa más ordenado y más seguro: el main solo dice encender o apagar y la clase decide qué nivel escribir, cuándo rechazar una orden y cómo arrancar apagada. También entendí que la parte de señal y la de potencia deben ir separadas y unidas solo por la tierra común, para no alimentar el motor desde el Arduino.